

PHASE 18 — CRYPTO-AGILITY, PQC/HYBRID MIGRATION PLANNER & AUTOMATED TRANSITION ENGINE

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

Theme: Blockchain & Cybersecurity

Phase: 18 of 22

Status: Migration Architecture, Crypto-Agility, PQC/Hybrid Transition & Verification

1. Purpose of This Phase

So far, ECDAT has answered:

What cryptography exists?

↓

Where is it?

↓

What depends on it?

↓

How risky is it?

↓

What should replace it?

Now we answer the hardest question:

How do we actually migrate?

This is where ECDAT evolves from a **discovery and assessment platform** into a **migration planning platform**.

The migration engine must take:

Current State

and produce:

Target State

+

Migration Path

+

Dependencies

+

Testing Strategy

+

Rollback Strategy

+

Verification

2. The Core Migration Problem

Replacing:

RSA

with:

PQC

is not equivalent to:

Find "RSA"

Replace "RSA" with "PQC"

Real systems contain:

Applications

Libraries

Protocols

Certificates

Keys

HSMs

KMS

Clients

Servers

Hardware

Firmware

APIs

Legacy dependencies

Changing one algorithm can affect many layers.

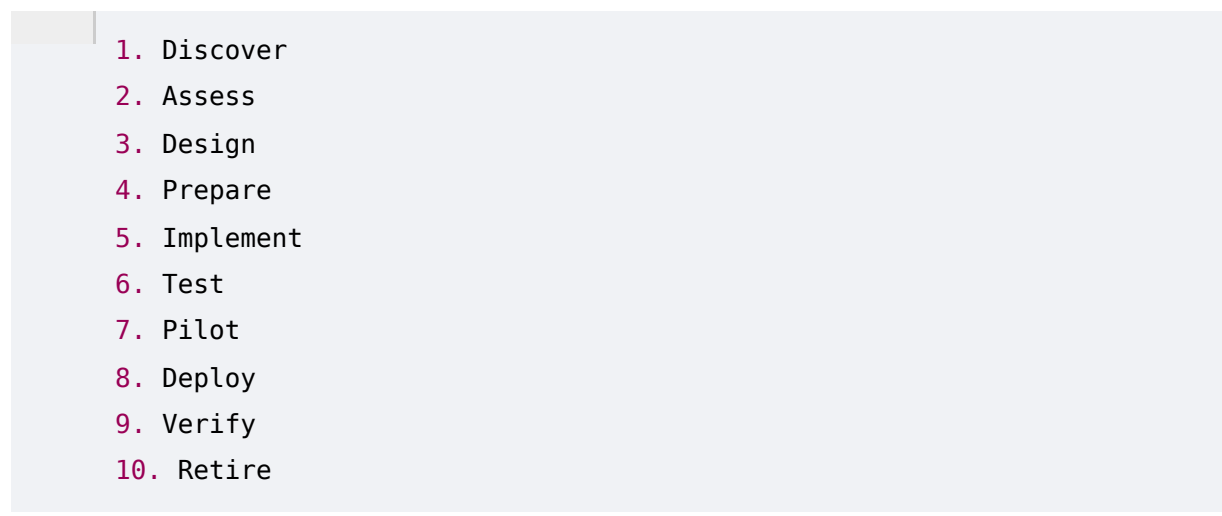
3. Migration Architecture

The complete migration engine:



4. Migration Is Not One Operation

ECDAT should treat migration as a lifecycle:

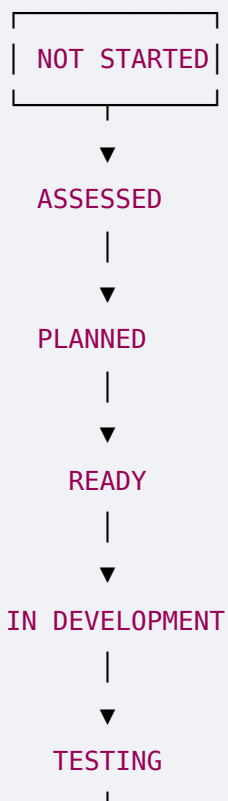


5. Migration States

Every asset should have a migration status.

UNKNOWN
NOT_STARTED
ASSESSED
PLANNED
READY
IN_DEVELOPMENT
TESTING
PILOT
ROLLOUT
VERIFICATION
MIGRATED
RETIRED
BLOCKED
EXCEPTION

6. Migration State Machine



Failure can move an asset into:

BLOCKED

7. What Is Crypto-Agility?

Crypto-agility means:

The ability of a system to change cryptographic algorithms, parameters, keys, providers, or protocols without redesigning the entire application.

This is one of the most important concepts in long-term PQC readiness.

8. Non-Agile Architecture

Example:

```
signature_algorithm = "ECDSA"
```

throughout the application.

Changing it requires:

```
Source changes
+
Testing
+
Deployment
+
Potential client changes
```

This is poor agility.

9. Agile Architecture

Instead:

```
signer = crypto_provider.get_signer(policy)
```

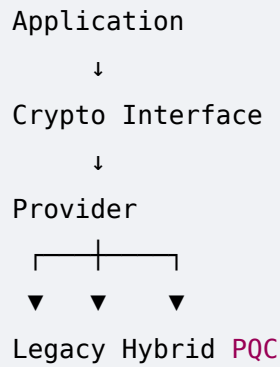
Configuration determines:

```
ECDSA
ML-DSA
Hybrid
```

The application doesn't hardcode the algorithm.

10. Crypto Provider Abstraction

Recommended pattern:



This allows algorithm replacement.

11. Algorithm Abstraction

Instead of:

```
if algorithm == "RSA":
```

prefer:

```
signature_provider.sign()
```

Then the provider determines the implementation.

12. Configuration-Driven Cryptography

Example:

```
cryptography:
  signature:
    policy: hybrid-primary

  encryption:
    policy: approved

  key_exchange:
    policy: hybrid
```

This allows migration policies to change without changing application logic.

13. Crypto Policy Layer

ECDAT should maintain an enterprise policy:

```
Algorithm
Parameter
Use Case
Environment
Approved Status
Target State
```

Example:

```
ECDSA P-256
Production
Approved legacy
Migration required
Target: PQC/hybrid
```

14. Why Policy Abstraction Matters

The application should ask:

```
| What signing mechanism should I use?
```

not:

```
| Should I use ECDSA or ML-DSA?
```

That decision belongs to the crypto policy layer.

15. Migration Target

Every vulnerable asset should eventually have:

```
Current Algorithm
Target Algorithm
Target Provider
Target Protocol
Target Key Type
Target Certificate
```

Example:

```
Current:
ECDSA P-256
```

Target:
Approved hybrid signature strategy

The exact target depends on application and protocol compatibility.

16. Recommendation ≠ Migration

This distinction is important.

Recommendation:

"Use a PQC/hybrid approach."

Migration plan:

1. Upgrade provider
2. Add target implementation
3. Enable dual support
4. Test compatibility
5. Deploy pilot
6. Monitor
7. Switch traffic
8. Retire legacy

17. Migration Planner

ECDAT should automatically generate a migration plan from the graph.

Input:

```
ECDSA
  ↓
Payment API
  ↓
OpenSSL
  ↓
HSM
  ↓
PKI
```

Output:

1. Validate HSM support
2. Upgrade crypto provider

3. Update **PKI**
4. Update application
5. Enable hybrid mode
6. Test clients
7. Pilot
8. Rollout

18. Dependency Ordering

Migration order matters.

For example:

Application

cannot migrate if:

HSM

does not support the target cryptography.

Therefore:

HSM

↓

Crypto Provider

↓

PKI

↓

Application

↓

Client

may be the correct order.

19. Migration Dependency Graph

Target Algorithm

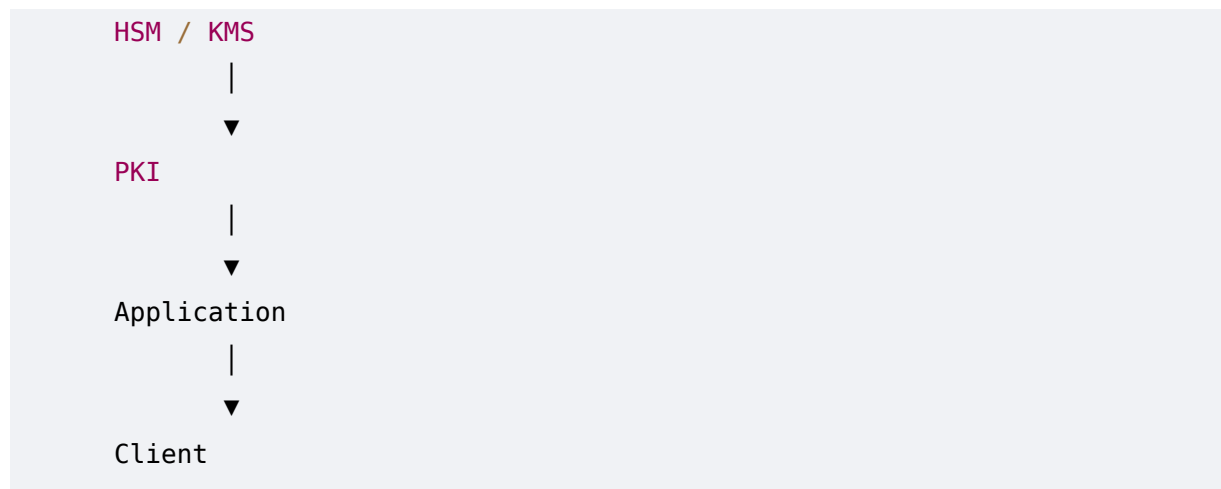
|

▼

Crypto Provider

|

▼



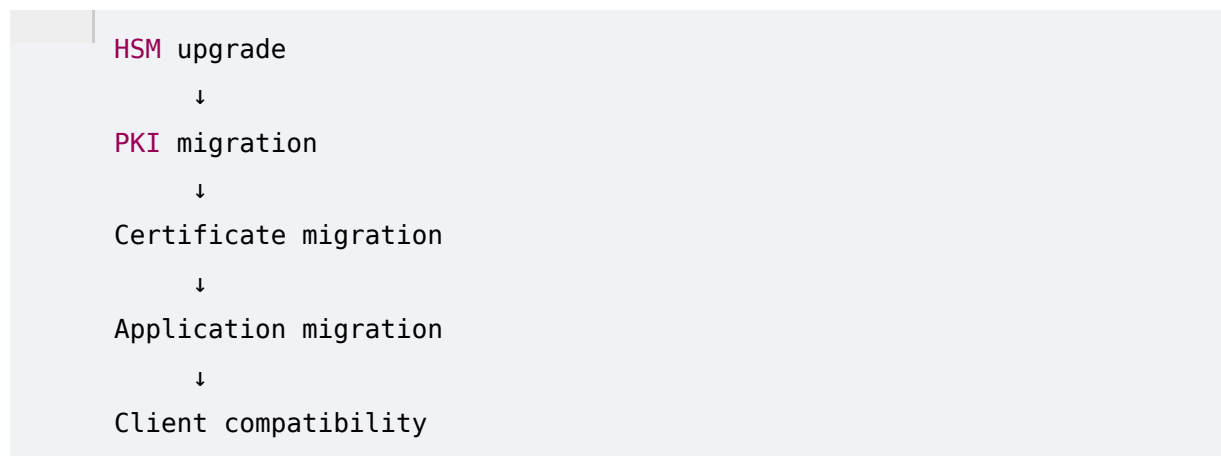
ECDAT should derive this from the knowledge graph.

20. Migration Critical Path

The planner identifies:

Longest dependency chain

Example:



This becomes the:

Migration Critical Path.

21. Migration Waves

Large organizations should not migrate everything at once.

Instead:

Wave 0:
Infrastructure preparation

Wave 1:

Low-risk applications

Wave 2:

Medium-risk applications

Wave 3:

Critical applications

Wave 4:

Legacy retirement

22. Pilot Group

Before enterprise-wide rollout:

Select:

3–10 representative applications

Include:

Simple

Medium

Complex

Legacy

High-volume

Critical

This exposes compatibility problems early.

23. Migration Readiness

An application can be classified:

READY

PARTIALLY READY

NOT READY

BLOCKED

based on:

```
Provider support
Protocol support
Library support
HSM support
Client support
Testing readiness
Crypto abstraction
```

24. Migration Readiness Score

Conceptually:

$$\lfloor \text{MRS} = w_1P + w_2L + w_3H + w_4C + w_5A \rfloor$$

where:

- $\lfloor (P) \rfloor$ = provider readiness
 - $\lfloor (L) \rfloor$ = library readiness
 - $\lfloor (H) \rfloor$ = hardware readiness
 - $\lfloor (C) \rfloor$ = compatibility readiness
 - $\lfloor (A) \rfloor$ = application agility
-

25. Example

```
Application:
Payment API

Provider:
Ready

HSM:
Ready

PKI:
Partial

Client:
Blocked

Crypto abstraction:
Ready
```

26. Compatibility Analysis

Before recommending a target algorithm, ECDAT should check:

```
Application
Library
Protocol
Client
Server
Certificate
HSM
KMS
Hardware
Operating System
```

27. Protocol Compatibility

Cryptographic migration often affects protocols.

Examples:

```
TLS
SSH
IPsec
VPN
Messaging
API authentication
```

ECDAT should identify protocol dependencies.

28. TLS Migration

A conceptual migration:

```
Current:
Classical key exchange

Target:
Hybrid key exchange
```

ECDAT should check:

- Server support
- Client support
- Load balancer support
- TLS library support
- Operating system support

29. Certificate Migration

Certificates are particularly complicated.

Migration may involve:

- Key generation
- CSR
- CA support
- Certificate issuance
- Deployment
- Client validation
- Trust store updates

Therefore certificate migration becomes a separate workflow.

30. Certificate Migration Workflow

```
graph TD; A[Generate Target Key] --> B[Generate CSR]; B --> C[CA Compatibility]; C --> D[Issue Certificate]; D --> E[Deploy]; E --> F[Validate]; F --> G[Monitor]; G --> H[Retire Old Certificate];
```

31. PKI Migration

Enterprise PKI may be the largest migration dependency.

ECDAT should model:

```
Root CA
Intermediate CA
Issuance Policy
Certificates
Applications
Trust Stores
```

32. Trust Store Problem

Even if the server supports the new algorithm:

```
Client
```

may reject the certificate because:

```
Trust Store
```

doesn't recognize the required chain.

ECDAT should therefore include client compatibility.

33. HSM Migration

HSMs can become major blockers.

ECDAT should assess:

```
Algorithm Support
Firmware
Provider
PKCS#11 support
API compatibility
Performance
Key lifecycle
```

34. HSM Migration Workflow

```
Inventory HSM
  ↓
Check target support
  ↓
Upgrade firmware/provider
  ↓
Test keys
  ↓
Test signing
  ↓
Pilot
  ↓
Production
```

35. KMS Migration

Cloud KMS can similarly require:

```
Key policy
Algorithm support
API changes
Rotation
Permissions
Application SDK changes
```

36. Key Migration

Never treat key migration as:

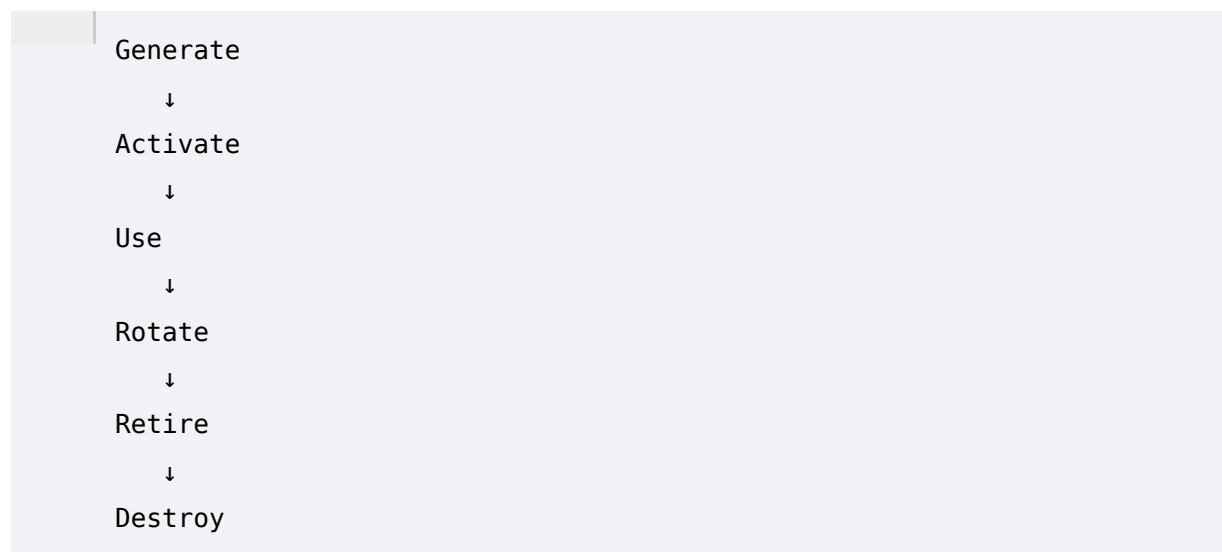
```
Copy private key
```

A secure migration may instead involve:

```
Generate new key
Provision securely
Update references
Rotate
Destroy old key
```

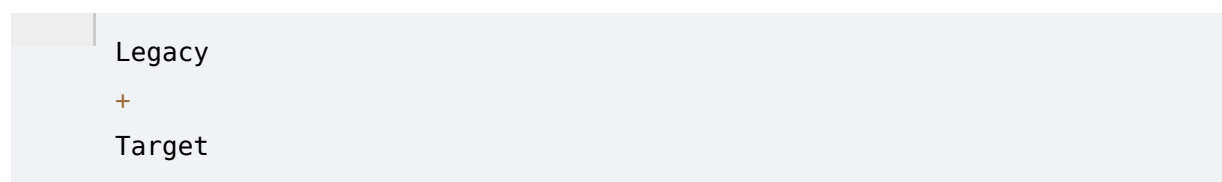
ECDAT should model the lifecycle.

37. Key Lifecycle



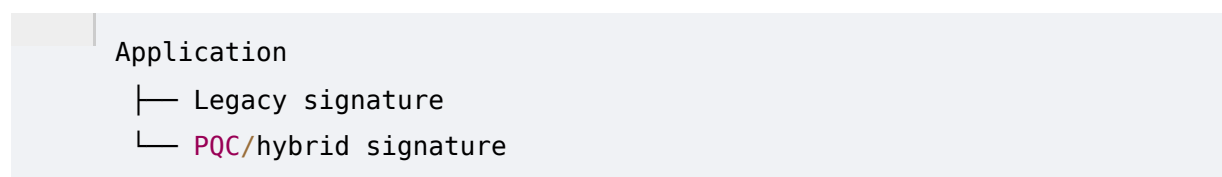
38. Dual-Key / Dual-Algorithm Strategy

For transition periods:



may coexist.

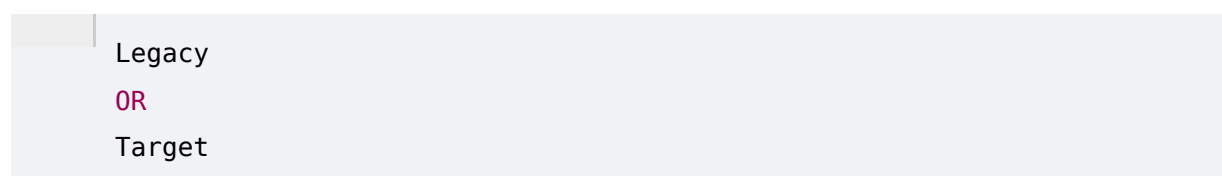
Example:



This supports gradual migration.

39. Dual Verification

A system may temporarily accept:



while producers transition.

This is useful for backward compatibility but introduces security and complexity considerations.

40. Migration Compatibility Matrix

ECDAT should display:

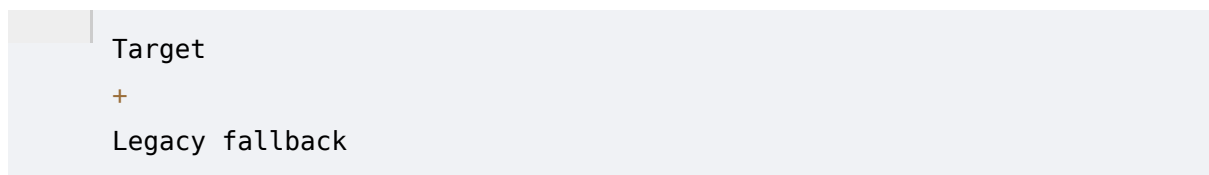
Component	Legacy	Target	Compatible?
Application	ECDSA	Hybrid	Yes
HSM	ECDSA	Target	Partial
Client A	ECDSA	Target	Yes
Client B	ECDSA	Target	No
PKI	Classical	Target	Partial

This immediately reveals blockers.

41. Backward Compatibility

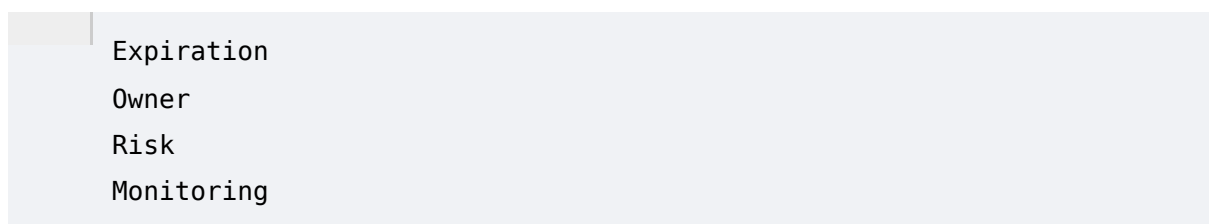
Legacy systems may need temporary support.

Therefore:



may be required.

But fallback should have:



42. Legacy Fallback

Example:

```
Fallback:
ECDSA

Allowed until:
2028-06-01

Owner:
Security Architecture

Reason:
Legacy client compatibility
```

ECDAT should automatically alert before expiration.

43. Canary Migration

Do not migrate every instance at once.

Instead:

```
1%
↓
5%
↓
25%
↓
50%
↓
100%
```

At each stage monitor:

```
Errors
Latency
Compatibility
Security events
```

44. Rollback

Every migration plan needs rollback.

Example:

```
Target enabled
↓
Errors increase
↓
Rollback to previous provider
↓
Investigate
```

45. Rollback Requirements

A migration is not production-ready unless:

```
Rollback exists
Rollback tested
Rollback owner exists
Rollback trigger defined
```

46. Migration Gates

Before each stage:

```
Gate 1:
Technical compatibility

Gate 2:
Security validation

Gate 3:
Performance validation

Gate 4:
Business approval

Gate 5:
Production deployment
```

47. Performance Testing

PQC algorithms may have different:

```
Key sizes
Signature sizes
CPU cost
Memory requirements
Network overhead
```

Therefore ECDAT should support benchmarking.

48. Performance Benchmark

Example:

```
Algorithm A
Sign: 1.2 ms
Verify: 0.8 ms

Target
Sign: 2.0 ms
Verify: 1.5 ms
```

The organization can determine whether the change is acceptable.

49. Latency Impact

ECDAT should estimate:

```
Baseline latency
Target latency
Difference
```

Example:

```
TLS handshake:
+18 ms
```

This may matter for latency-sensitive applications.

50. Network Overhead

Some cryptographic schemes can change message sizes.

ECDAT should assess:

- Signature Size
- Key Size
- Handshake Size
- Packet Size

especially for:

- IoT
- Mobile
- Embedded
- High-throughput
- Low-bandwidth

systems.

51. Resource Constraints

For constrained systems:

- CPU
- RAM
- Flash
- Network
- Battery

matter.

Migration planning should consider hardware constraints.

52. Embedded Systems

Embedded firmware may be especially difficult.

Graph:

```
graph TD; Firmware --> CryptoLibrary[Crypto Library]; CryptoLibrary --> Algorithm; Algorithm --> HardwareAccelerator[Hardware Accelerator];
```

- Firmware
- ↓
- Crypto Library
- ↓
- Algorithm
- ↓
- Hardware Accelerator

Migration may require:

Firmware Update
Hardware Change

53. Vendor Dependencies

Some systems cannot migrate until a vendor provides support.

ECDAT should track:

Vendor
Product
Version
Target Support
Expected Availability

54. Vendor Blocker

Example:

Application:
Legacy ERP
PQC support:
Vendor roadmap
Status:
BLOCKED
Reason:
Vendor software does not support target cryptography.

55. Migration Effort Estimation

ECDAT can estimate:

Low
Medium
High
Very High

based on:

```
Code changes
Dependencies
Testing
Infrastructure
Clients
Certificates
```

56. Effort Model

Conceptually:

$$E = C + D + I + T + V$$

where:

- C = code changes
 - D = dependency complexity
 - I = infrastructure work
 - T = testing
 - V = vendor dependency
-

57. Migration Risk

Migration itself has risk.

ECDAT should distinguish:

```
Current Security Risk
```

from:

```
Migration Risk
```

Example:

```
Current risk:
High
```

```
Migration risk:
Medium
```

58. Migration Decision Matrix

	Low Migration Risk	High Migration Risk
High Security Risk	Immediate	Strategic program
Low Security Risk	Quick win	Defer / plan

This avoids simplistic prioritization.

59. Migration Planner Output

For every asset:

```

Current State
Target State
Dependencies
Blockers
Steps
Owner
Effort
Risk
Timeline
Rollback
Verification

```

60. Example Migration Plan

```

APPLICATION:
Payment API

CURRENT:
ECDSA P-256

```

TARGET:

Approved hybrid signature strategy

OWNER:

Payments Team

RISK:

Critical

MIGRATION EFFORT:

High

Steps:

1. Validate crypto provider
2. Validate HSM
3. Update PKI
4. Implement abstraction
5. Enable target in test
6. Run compatibility tests
7. Pilot 1%
8. Monitor
9. Increase rollout
10. Verify
11. Retire legacy

61. Automated Plan Generation

ECDAT should be able to generate this from:

CBOM

- + Knowledge Graph
- + Risk
- + Policy
- + Recommendation

62. AI's Role in Migration Planning

AI can help generate:

```
Migration explanation
Test scenarios
Potential blockers
Developer tasks
Documentation
```

But deterministic systems should control:

```
Target algorithm eligibility
Policy
Risk
Dependencies
Approval
```

63. AI-Generated Migration Tasks

Example:

```
Task 1
Replace direct ECDSA dependency with crypto abstraction.

Task 2
Upgrade provider to approved version.

Task 3
Validate HSM compatibility.

Task 4
Add hybrid verification.

Task 5
Create compatibility tests.
```

64. Task Generation From Evidence

AI should reference actual evidence:

```
src/security/sign.py
```

rather than inventing:

```
src/authentication/crypto.py
```

if it doesn't exist.

65. Developer Migration Workspace

ECDAT could provide:

```
Finding
↓
Migration Plan
↓
Tasks
↓
Code Locations
↓
Testing
↓
Verification
```

66. Migration Tickets

Generate:

```
Title
Description
Priority
Owner
Dependencies
Acceptance Criteria
Evidence
```

67. Acceptance Criteria

Example:

```
[ ] No direct ECDSA API usage
[ ] Target provider enabled
[ ] Unit tests pass
[ ] Integration tests pass
[ ] Performance benchmark passes
```

- [] Production scan confirms target
- [] Legacy usage absent or approved exception

68. Automated Verification

After deployment:

```
Scan
↓
Compare
↓
Expected State
vs
Observed State
```

69. Verification Rules

Example:

```
Expected:
No active RSA-1024

Observed:
0

Status:
PASS
```

70. Migration Regression

Suppose migration succeeds:

```
ECDSA → PQC
```

Then a later release reintroduces ECDSA.

ECDAT detects:

```
Crypto Regression
```

and alerts.

71. Regression Detection

Previous:

PQC

Current:

PQC + ECDSA

Policy:

Legacy not allowed

Result:

REGRESSION

72. Continuous Migration Monitoring

Migration is not a one-time activity.

ECDAT should continuously check:

New Legacy Usage

New Certificates

New Dependencies

Policy Violations

Crypto Drift

73. Crypto-Agility Score

ECDAT can measure how easy a system is to migrate.

Possible factors:

Algorithm hardcoding

Provider abstraction

Configuration-driven crypto

Centralized policy

Dependency isolation

Automated testing

PQC support

74. Example

Application A
Crypto Agility: 82

Application B
Crypto Agility: 31

Application B should likely receive engineering investment before its next migration.

75. Crypto-Agility Maturity Levels

Level 0:
Hardcoded

Level 1:
Partial abstraction

Level 2:
Provider abstraction

Level 3:
Policy-driven

Level 4:
Dynamic algorithm selection

Level 5:
Continuous cryptographic agility

76. Architecture Assessment

ECDAT can identify:

Hardcoded Algorithm

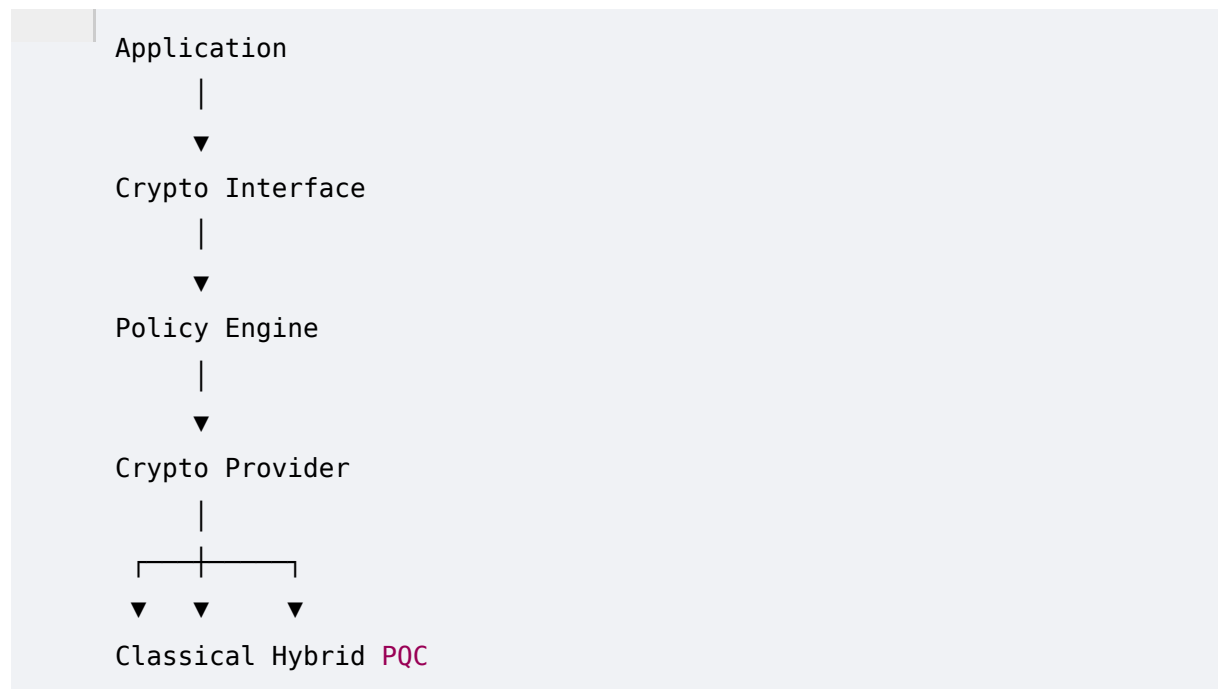
and recommend:

Introduce a crypto abstraction layer before attempting PQC migration.

This is often more valuable than directly recommending an algorithm.

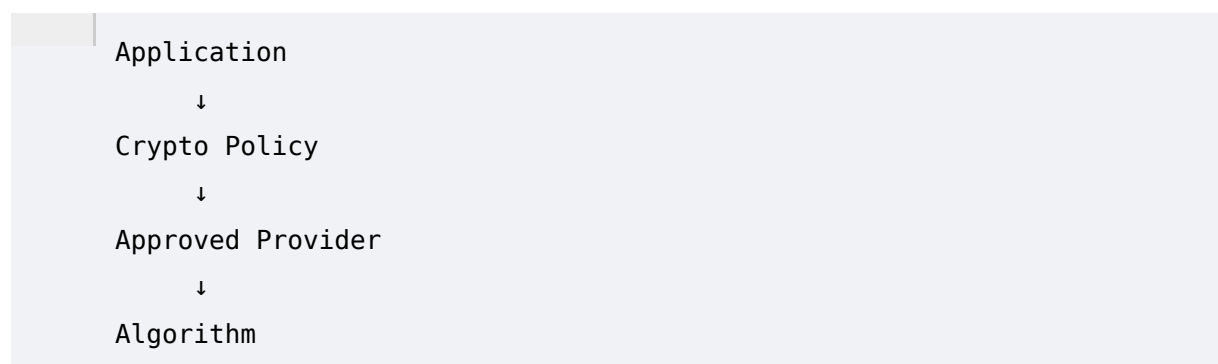
77. Migration Architecture Pattern

Recommended:



78. Long-Term Architecture

The goal is:

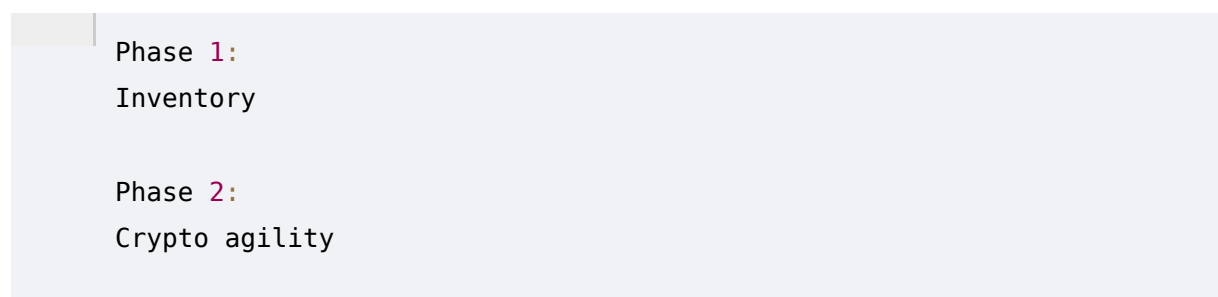


The application should not be tightly coupled to the algorithm.

79. PQC Strategy

PQC migration should be staged.

Potential model:



Phase 3:
PQC capability

Phase 4:
Hybrid deployment

Phase 5:
PQC-first

Phase 6:
Legacy retirement

80. Why Hybrid Matters

Hybrid mechanisms can allow organizations to maintain classical compatibility while introducing post-quantum protection, where supported by the relevant protocol and standards.

This can reduce transition risk.

81. Hybrid Strategy

Conceptually:

Classical
+
PQC

rather than immediately:

Classical
→
PQC only

But ECDAT should not assume hybrid is always the right answer.

82. Target Selection Factors

The recommendation engine should consider:

Security
Protocol
Performance

```
Compatibility
Library Support
Hardware Support
Key Size
Signature Size
Latency
Cost
Vendor Support
Migration Effort
```

83. Algorithm Selection

The system should select from:

```
Approved Algorithm Registry
```

rather than allowing unrestricted AI suggestions.

84. Algorithm Registry

Example structure:

```
Algorithm
Family
Status
Approved Use Cases
Disallowed Use Cases
Security Level
Implementation Availability
```

85. Migration Registry

ECDAT should also track:

```
Algorithm
Current State
Target State
Migration Pattern
Compatibility
```

Example:

ECDSA

→

Hybrid Signature Pattern

86. Migration Pattern Library

Reusable migration patterns:

Legacy → Hybrid

Legacy → PQC

Certificate Rotation

Provider Upgrade

Library Upgrade

HSM Upgrade

Crypto Abstraction

Protocol Upgrade

87. Reusable Migration Plans

If 500 applications use the same pattern:

ECDSA

+

OpenSSL

+

HSM

ECDAT can reuse:

Migration Template

rather than generating 500 completely independent plans.

88. Migration Template

```
pattern: ecdsa-to-hybrid
```

```
requirements:
```

- provider_support
- hsm_support
- pki_support

```
steps:
  - upgrade_provider
  - enable_hybrid
  - test
  - pilot
  - rollout
  - verify
```

89. Migration Factory

This leads to an enterprise concept:

Migration Factory.

Instead of migrating applications one by one manually:

```
Pattern
↓
Reusable Workflow
↓
Many Applications
```

90. Migration Factory Dashboard

```
Migration Factory

Pattern: ECDSA → Hybrid

Eligible:
428 applications

Completed:
72

In progress:
91

Blocked:
34
```

91. Batch Migration

Applications can be grouped by:

- Technology
- Team
- Business Unit
- Algorithm
- Library
- Dependency
- Risk

Then migrated in batches.

92. Migration Portfolio

Example:

Portfolio:
Public Key Migration

Applications:
1,284

Total risk:
High

Target:
PQC / Hybrid

Completion:
31%

93. Migration Budgeting

ECDAT can eventually estimate:

- Engineering effort
- Infrastructure cost
- Vendor cost

Testing effort
Downtime risk

This enables planning.

94. Cost Estimation

Conceptually:

$$\text{Cost} = \text{Engineering} + \text{Infrastructure} + \text{Vendor} + \text{Testing} + \text{Operations}$$

The estimates should be configurable and clearly labeled as estimates.

95. Investment Prioritization

Combine:

Risk Reduction
+
Blast Radius
+
Migration Effort
+
Cost

to prioritize investments.

96. Migration ROI

Conceptually:

$$\text{ROI} = \frac{\text{Risk Reduction Value}}{\text{Migration Cost}}$$

This allows executive comparison of initiatives.

97. Migration Program Governance

Each program should have:

Owner
Sponsor
Security Architect
Engineering Lead
Infrastructure Lead
Milestones

Risk
Budget

98. Approval Workflow

Potential stages:

```
Draft
↓
Security Review
↓
Architecture Review
↓
Business Approval
↓
Implementation
↓
Verification
```

99. Change Management

Migration changes should integrate with enterprise change-management systems where appropriate.

ECDAT should record:

```
Change ID
Deployment Date
Owner
Scope
Result
Rollback
```

100. Migration Evidence

Every completed migration should preserve:

```
Before State
Migration Plan
Changes
Tests
Deployment
```

After State
Verification

This becomes audit evidence.

101. Migration Certificate

ECDAT could produce a verification record:

```
APPLICATION:
Payment API

OLD:
ECDSA P-256

NEW:
Approved hybrid strategy

VERIFICATION:
Passed

DATE:
...

SCAN:
...
```

102. Cryptographic Attestation

In a mature system, ECDAT could eventually generate machine-readable attestation:

```
{
  "application": "payment-api",
  "migration_status": "verified",
  "legacy_assets_remaining": 0,
  "target_assets_detected": 4,
  "verified_at": "..."
```

103. Migration Rollback Evidence

If rollback occurs:


```
Migration:
Failed

Reason:
Legacy client incompatibility

Rollback:
Successful

Next Action:
Client upgrade
```

The system should retain this history.

104. Migration Learning Loop

Every migration teaches ECDAT:

```
Estimated effort
vs
Actual effort
```

Then future estimates can improve.

105. Effort Prediction

Suppose ECDAT estimated:

```
20 engineer-days
```

Actual:

```
32 engineer-days
```

It records the difference.

Over time:

```
Historical Data
↓
Better Estimation
```

106. AI Migration Learning

AI can identify recurring blockers:

```
HSM incompatibility
Legacy clients
Vendor delay
Certificate limitations
```

Then future plans can warn:

```
Similar applications historically encountered HSM compatibility issues.
```

107. Migration Risk Forecasting

ECDAT can predict:

```
Potential blockers
Potential effort
Potential delay
```

but should label these as forecasts, not certainties.

108. Phase 18 End-to-End Example

Consider:

```
Application:
Payment API

Current:
ECDSA P-256

Risk:
High

Dependency:
OpenSSL

HSM:
Legacy
```

ECDAT generates:

TARGET:

Approved hybrid strategy

Then:

STEP 1

Upgrade provider

STEP 2

Validate HSM

STEP 3

Update PKI

STEP 4

Introduce crypto abstraction

STEP 5

Enable target alongside legacy

STEP 6

Run compatibility tests

109. What the User Sees

PAYMENT API MIGRATION

Current: ECDSA P-256

Target: Approved Hybrid Strategy

Risk: HIGH

Readiness: 68%

Blockers:

- Legacy HSM
- 2 legacy clients

Estimated effort: High

110. Phase 18 Requirements

ECDAT's migration engine should support:

Architecture

- Crypto abstraction
- Provider abstraction
- Policy-driven algorithms
- Configuration-driven cryptography

Migration

- Current state
- Target state
- Dependencies
- Migration waves
- Critical path
- Pilot
- Rollout
- Rollback

Compatibility

- Applications
- Libraries
- Protocols
- Certificates
- PKI
- HSM
- KMS
- Clients
- Hardware

Verification

- Rescan
- Compare
- Regression detection

- Evidence
- Attestation

Governance

- Ownership
- Approvals
- Exceptions
- Audit
- Change management

111. The Most Important Principle

ECDAT should never say:

"Replace RSA with Algorithm X."

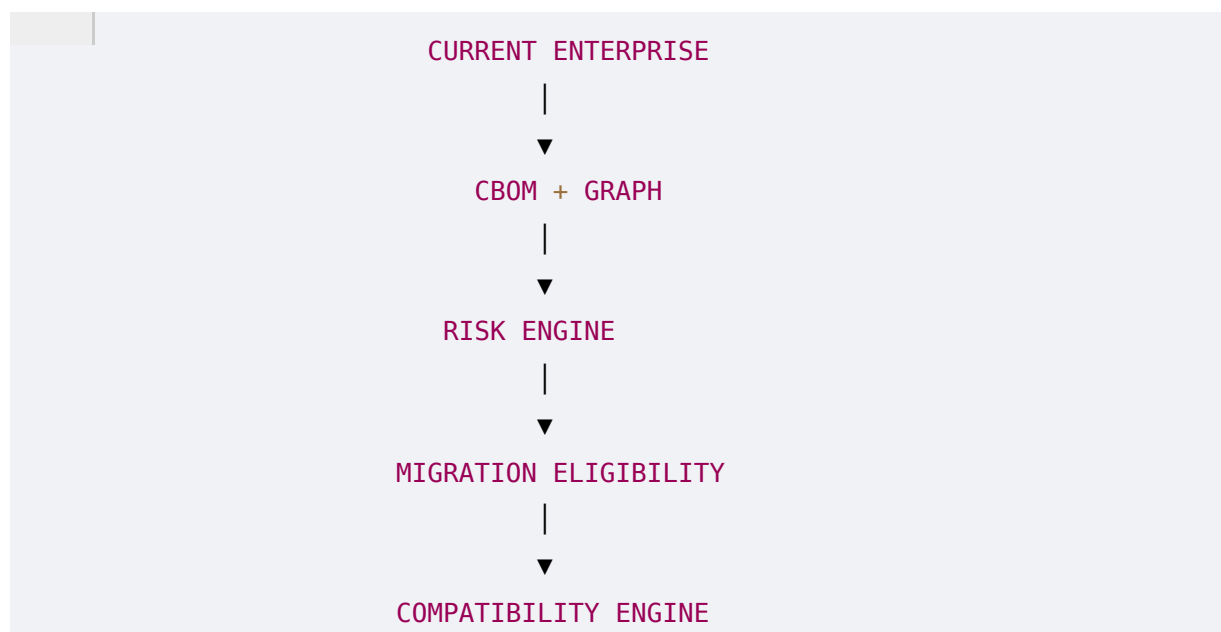
and stop there.

It should say:

"Here is the current cryptographic dependency, here is why it is risky, here is the approved target state, here are the blockers, here is the safest migration sequence, here is the rollback strategy, and here is how we will verify that the migration actually happened."

That is a fundamentally more enterprise-grade solution.

112. Phase 18 Final Architecture



↓
TARGET SELECTION
↓

113. Phase 18 Final Takeaway

The ultimate goal of ECDAT is not:

Find vulnerable cryptography.

It is:

Find
↓
Understand
↓
Prioritize
↓
Plan
↓
Migrate
↓
Verify
↓
Prevent Regression

And the long-term architectural destination is:

Crypto-agility as an enterprise capability.

Instead of repeatedly performing painful migrations whenever cryptography changes, the organization should eventually reach a state where:

Algorithm changes
↓
Policy changes
↓
Provider changes
↓
Application adapts

rather than:

Algorithm changes



Hundreds of applications require rewrites

That is the real strategic value of the ECDAT platform.

PHASE 19 PREVIEW — ENTERPRISE SECURITY, ZERO-TRUST ARCHITECTURE, DATA PROTECTION & DEPLOYMENT MODEL

The next phase tackles an extremely important issue:

ECDAT itself will be handling some of the most sensitive information inside an organization.

The platform may see:

Source code
Crypto configurations
Certificates
Infrastructure topology
HSM metadata
KMS metadata
Application architecture
Business criticality
Security findings
Migration plans

So the next question is:

How do we make ECDAT itself secure enough to deploy inside a high-security enterprise environment?

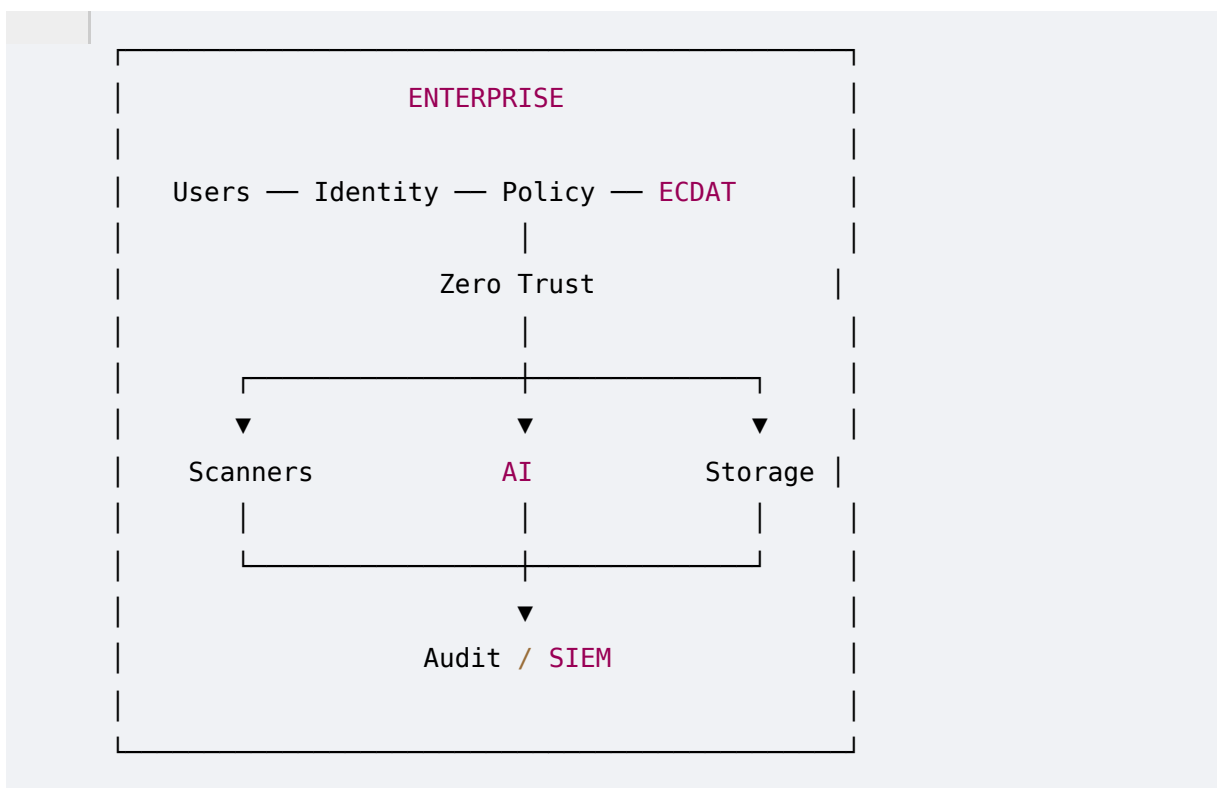
Phase 19 will cover:

- Zero-trust architecture
- Identity
- Authentication
- Authorization

- RBAC
- ABAC
- Service identity
- mTLS
- API security
- Secrets management
- Key management
- Encryption at rest
- Encryption in transit
- Tenant isolation
- Data classification
- Source-code protection
- Private deployment
- Air-gapped deployment
- On-prem deployment
- Hybrid deployment
- Secure AI
- Prompt injection defense
- Model isolation
- AI data boundaries
- Scanner sandboxing
- Container isolation
- Supply-chain security
- Secure CI/CD
- SBOM
- Artifact signing
- Dependency verification
- Audit logging
- Tamper resistance
- SIEM integration
- Incident response
- Backup
- Disaster recovery

- High availability
- Secure updates
- Offline updates
- Vulnerability management
- Platform hardening
- Threat modeling
- Attack surface analysis
- Security testing
- Penetration testing
- Red teaming
- Compliance architecture
- Deployment topology

The central idea will be:



The objective is to make ECDAT not only **intelligent**, but also **deployable and trustworthy** in the kind of **high-security environment** the problem statement is actually targeting.

Phase 18 complete.